คู่มือเตรียมสัมภาษณ์ Senior Data Architect (DDIA x Omni-Flow Labs)

เอกสารนี้เป็นแนวทางเตรียมตัวสัมภาษณ์ตำแหน่ง Senior Data Architect โดยเชื่อมโยงแนวคิดสำคัญจากหนังสือ Designing Data-Intensive Applications (DDIA) ของ Martin Kleppmann ทั้ง 12 บท เข้ากับประสบการณ์จริงจากโปรเจกต์ Omni-Flow Labs ชุดการทดลอง (Labs) เพื่อใช้ตอบคำถามสัมภาษณ์เชิงลึกระดับ Senior/Toptal ได้อย่างมั่นใจ ครอบคลุมทั้งคำถามสำคัญในแต่ละบท พร้อมคำตอบที่วิเคราะห์ ข้อดี-ข้อด้อย และ trade-off ของเทคโนโลยีและสถาปัตยกรรมข้อมูลที่เกี่ยวข้อง

บทสรุปผู้บริหาร

- เนื้อหาโดยสังเขป: สรุปแนวคิดหลักจาก DDIA ทั้ง 12 บท เช่น Reliability, Scalability, Data Models, Storage Engines, Transactions, Distributed Systems, Consistency Models, Batch/Stream Processing และ Data Integration พร้อมโยงเข้ากับตัวอย่างจริงใน Omni-Flow Labs เพื่อแสดงวิธีใช้งานและทดสอบทฤษฎีด้วยระบบจริง
- รูปแบบการศึกษา: นำเสนอในลักษณะ แนวคำถาม-คำตอบ (Q&A) ที่ผู้สัมภาษณ์มักถามและคาดหวังคำอธิบายที่ลึกซึ้ง เจาะประเด็น trade-off ต่าง ๆ ให้พร้อมโต้ตอบ กรณีผู้สัมภาษณ์ถามจี้รายละเอียด
- การเน้นย้ำ:
  - การอธิบาย คุณสมบัติระบบข้อมูล ที่สำคัญ เช่น ความน่าเชื่อถือ (Reliability) การขยายระบบ (Scalability) ความคงทน (Durability) ความสอดคล้องของข้อมูล (Consistency) และปัจจัยอื่น ๆ โดยไม่ใช้คำที่คลุมเครือ แต่ระบุขอบเขตของการรับประกันให้ชัดเจน เช่น exactly-once ในระบบไหน อย่างไร
  - ตัวอย่างจาก Omni-Flow Labs เช่น การทดสอบ Fault Tolerance (ระบบไม่ล่มเม้องค์ประกอบบางส่วนขัดข้อง) การทดสอบ Throughput และ Latency ด้วยภาระงานสูง (เช่น 1,500 TPS), การแก้ปัญหา hot key ด้วย key salting, การใช้ Lakehouse และ CDC ฯลฯ
  - การตรวจสอบคุณภาพข้อมูล (Data Quality) ผ่านกระบวนการ Data Reconciliation & Audit การนับตรวจสอบข้อมูลระหว่างระบบต้นทาง-ปลายทางเพื่อให้มั่นใจว่า ไม่มีข้อมูลสูญหายหรือซ้ำซ้อน และการกำหนด Data Quality Gate เพื่อกรองข้อมูลที่ไม่ผ่านเกณฑ์เข้า Quarantine

การใช้คู่มือนี้: ผู้อ่านสามารถใช้ชุด Q&A ด้านล่างนี้เพื่อทบทวนความเข้าใจของตนเอง รวมถึงจำลองสถานการณ์ตอบคำถามในการสัมภาษณ์ โดยเน้นให้อธิบายในภาพกว้างของหลักการก่อนเจาะประเด็นเชิงลึกเสมอ และควรเตรียมตัวอ้างอิงถึงตัวอย่างจากประสบการณ์จริง (เช่น Labs ใน Omni-Flow) เพื่อแสดงให้เห็นถึงความเข้าใจเชิงปฏิบัติ

คู่มือถาม-ตอบ (Q&A Cheat Sheet) – แนวคำถามสัมภาษณ์ 12 หัวข้อสำคัญ

ด้านล่างนี้คือชุดคำถามที่มักพบในการสัมภาษณ์ ที่ครอบคลุมหัวข้อในแต่ละบทของหนังสือ DDIA พร้อมตัวอย่างคำตอบที่สรุปจากทั้งหลักการในหนังสือและประสบการณ์ในโปรเจกต์ Omni-Flow Labs

บทที่ 1: Reliable, Scalable, Maintainable Applications (ระบบเชื่อถือได้ ขยายได้ ดูแลรักษาง่าย)

Q: อะไรคือคุณสมบัติหลักของ "ระบบที่เน้นข้อมูล" (data-intensive application) ที่ดี? A: ระบบข้อมูลที่ดีต้องมี 3 คุณสมบัติหลัก ได้แก่ ความน่าเชื่อถือ (Reliability), ความสามารถในการขยาย (Scalability), และการดูแลรักษาง่าย (Maintainability) (www.bookey.app) (www.bookey.app)

- ความน่าเชื่อถือ (Reliability): ระบบจะต้องทำงานให้ถูกต้องต่อเนื่อง แม้มีส่วนใดส่วนหนึ่งเกิดปัญหา (fault) ก็ตาม ไม่ทำให้บริการโดยรวมล่ม (no single point of failure)
- ความสามารถในการขยาย (Scalability): ระบบสามารถรองรับ โหลดงานที่เพิ่มขึ้น ได้ โดยการเพิ่มประสิทธิภาพหรือเพิ่มจำนวนทรัพยากร (scale up/out) โดยที่ performance ลดลงไม่มาก (มักดูจาก throughput และ latency เป็นหลัก)
- การดูแลรักษาง่าย (Maintainability): โค้ดและสถาปัตยกรรมของระบบควรเรียบง่าย แก้ไขปรับปรุงได้ง่าย และรองรับการเปลี่ยนแปลงความต้องการธุรกิจในอนาคตโดยไม่สร้าง technical debt มากเกินไป

Q: ระหว่าง fault กับ failure ต่างกันอย่างไร? ทำไมการแยกสองคำนี้จึงสำคัญในระบบกระจาย? A: ในบริบทของความน่าเชื่อถือ เราจำเป็นต้องแยกให้ออกระหว่าง Fault กับ Failure (university.kurz.fyi):

- Fault (จุดขัดข้อง) หมายถึง ส่วนประกอบย่อย ของระบบเกิดความผิดปกติ เช่น เซิร์ฟเวอร์เครื่องหนึ่งล่ม, แรมของฐานข้อมูลเสีย, หรือมีบั๊กใน service ใด service หนึ่ง แต่ ระบบโดยรวมยังไม่หยุดทำงาน
- Failure (ระบบล้มเหลว) คือ ระบบโดยรวมไม่สามารถให้บริการตามที่ควรจะเป็น (เช่น response ผิดหรือไม่ตอบสนองผู้ใช้) เมื่อเกิดความผิดพลาดที่ไม่ได้ถูกกักไม่ให้ลุกลามจากจุดขัดข้องเล็ก ๆ

ดังนั้น เป้าหมายของการออกแบบที่ดี คือทำให้ fault ใด ๆ ไม่ลุกลามเป็น failure ได้ ผ่านวิธีการอย่าง redundancy (ระบบสำรอง) การ retry, การทำ isolation และ fault-tolerant design อื่น ๆ (university.kurz.fyi) (university.kurz.fyi)

Q: เราจะวัดความสามารถในการรองรับโหลด (Scalability) ของระบบได้อย่างไร? A: ให้พิจารณาว่า เมื่อเพิ่มภาระงาน (เช่น จำนวน request/ผู้ใช้งาน) ผลกระทบต่อ performance metrics เช่น latencies หรือ throughput เป็นอย่างไร

- Throughput (ปริมาณงานต่อวินาที): จำนวนงาน (เช่น requests หรือธุรกรรม) ที่ระบบประมวลผลได้ต่อหน่วยเวลา
- Latency (ความหน่วง): เวลาที่ระบบใช้ตอบสนองงานหนึ่ง ๆ โดยมักรายงานเป็นค่า เปอร์เซ็นไทล์ (percentile) แทนที่จะใช้ค่าเฉลี่ย เพราะการดู percentile (เช่น p95, p99) ช่วยบอก ประสบการณ์แย่สุดของผู้ใช้กลุ่มเกือบสูงสุด (worst-case) ได้ (www.bookey.app)
- ในการสัมภาษณ์ ผมจะยกตัวอย่าง Omni-Flow Command Center ซึ่งมีส่วนที่ใช้ทดสอบ throughput และ latency: เช่นการส่งข้อมูลเข้า Kafka 1,500 ข้อความต่อวินาที แล้วดู p50/p95/p99 latency ว่าหาก throughput เพิ่มขึ้น ค่า p99 latency เพิ่มขึ้นแค่ไหน เป็นต้น เพื่อดูว่าระบบยังรองรับผู้ใช้เพิ่มได้หรือไม่โดย latency ยังอยู่ในเกณฑ์ที่ยอมรับได้

Q: ใน Omni-Flow labs มีการทดสอบความน่าเชื่อถือและการขยายระบบอย่างไร? A: ตัวอย่างจาก Omni-Command Center (Lab หลัก):

- การทดสอบ Fault Tolerance: Scenario 2 – ปิดการทำงานของ consumer (จำลอง network partition หรือ consumer crash) เป็นเวลา 3 วินาที แล้วสังเกตว่า producer และระบบ Kafka ยังคงส่งและเก็บข้อความต่อไปได้หรือไม่ (ไม่มี message สูญหาย) และเมื่อ consumer กลับมา ระบบสามารถตามอ่านข้อความทั้งหมดทัน (ไม่มี message ตกหล่น)
- การทดสอบ Load & Performance: Scenario 4 – รัน stress test โดยยิง event ในอัตราสูง (เช่น ~1,500 ข้อความ/วินาที) เข้า Kafka แล้วตรวจสอบ throughput ของ consumer และค่าความหน่วง (latency) โดยเฉพาะ p99 latency ว่ายังอยู่ในระดับที่ยอมรับ (เช่น < 500 ms) หรือไม่ เพื่อยืนยันว่าเมื่อเพิ่มโหลด ระบบยัง scalable (การเพิ่ม throughput ไม่ทำให้ latency สูงเกินไป)

Q: หากสถาปัตยกรรมรองรับ throughput เพิ่มขึ้น แต่ p99 latency แย่ลงเรื่อย ๆ เราควรวิเคราะห์สิ่งใดก่อน? A: เบื้องต้นต้องดูว่าเกิด bottleneck ที่ส่วนใดของระบบ ซึ่งอาจแยกพิจารณาได้ดังนี้:

- ที่ฝั่งแหล่งข้อมูล (Producer): ดูว่า Producer สามารถ ส่งข้อมูลถึง throughput ที่ต้องการโดยไม่หน่วงเกินหรือไม่ (Ex: CPU สูง ติดขัดที่ network bandwidth ฯลฯ)
- ที่ระบบกลาง (Broker/Kafka): ตรวจดู latency per message ใน Kafka, ตรวจสอบ consumer lag และการเกิด backpressure – ถ้า consumer อ่านไม่ทัน อาจมี messages รอใน broker ข้าม retention period จนถูกลบ
- ที่ปลายทาง (Consumer/Sink): ตรวจสอบ consumer throughput, error rate และ processing time ของแต่ละ event มี outlier หรือ event ใดกินเวลานานกว่าปกติหรือไม่

ในการแก้ปัญหา p99 latency สูง อาจต้องเพิ่มทรัพยากร (scale out) ตรงจุด bottleneck หรือปรับปรุงโค้ดให้มีประสิทธิภาพขึ้น รวมถึง circuit breaker/backpressure ป้องกันไม่ให้โหลดเกินกำลังของระบบ

บทที่ 2: Data Models and Query Languages (แบบจำลองข้อมูลและภาษาสอบถาม)

Q: อธิบายความแตกต่างระหว่าง Relational, Document และ Graph Data Model และกรณีการใช้งานที่เหมาะสมของแต่ละแบบ A:

- Relational model (เชิงสัมพันธ์): จัดเก็บข้อมูลลงในตาราง (rows/columns) แต่ละ Record เชื่อมกันด้วย Foreign Key มี Schema ที่ชัดเจนล่วงหน้า เหมาะกับข้อมูลโครงสร้างชัด มีความสัมพันธ์ที่ ตายตัว และต้องการ ACID transactions (เช่น ระบบการเงิน ธุรกรรม)
- Document model (เอกสาร): เก็บข้อมูลเป็น JSON/XML หรือแบบ Key-value ที่ ยืดหยุ่น แต่ละ record สามารถมีโครงสร้าง nested และแตกต่างกันได้ เหมาะกับข้อมูลที่โครงสร้างไม่แน่นอนหรือเปลี่ยนแปลงบ่อย (เช่น NoSQL, MongoDB) อย่างไรก็ตามการ Query ข้ามเอกสารหรือการทำ join หลาย collection จะยากกว่า และอาจต้องซ้ำข้อมูลระหว่างเอกสาร
- Graph model (กราฟ): เก็บข้อมูลแบบโหนด (nodes) และความสัมพันธ์ (edges) เหมาะกับข้อมูลที่ความสัมพันธ์ระหว่าง entity มีความสำคัญสูง เช่น social network หรือ recommendation engine โดย Query สามารถเดินตามกราฟ (traverse) ได้มีประสิทธิภาพ

การเลือก Data Model ควรเริ่มจากลักษณะ workload และ query patterns ของเรา: ถ้าต้องการความถูกต้องของข้อมูลสูง และ join บ่อย Relational, ถ้าต้องการ ความยืดหยุ่น และ scale out ง่าย Document/NoSQL, ถ้าข้อมูลเป็น network ของความสัมพันธ์ซับซ้อน Graph

Q: Schema-on-read vs Schema-on-write ต่างกันอย่างไร? A:

- Schema-on-write (schema แบบกำหนดล่วงหน้า): เน้น บังคับใช้ schema ขณะเขียนข้อมูล – Data ที่เขียนเข้าระบบต้องถูกแปลงให้อยู่ในโครงสร้างที่สอดคล้องกับ schema ที่กำหนดไว้ก่อนแล้ว ข้อดี: ข้อมูลมีความสะอาดและสม่ำเสมอ สูงเพราะถูก validate ตั้งแต่เขียน ข้อด้อย: ถ้า schema เปลี่ยนต้อง migrate ข้อมูลที่มีอยู่, อาจไม่ยืดหยุ่นกับ data ชนิดใหม่
- Schema-on-read (schema กำหนดทีหลัง): ไม่บังคับใช้ schema ตอนเขียน, เก็บข้อมูลดิบไว้ แล้วค่อยกำหนด schema เมื่ออ่าน (ผู้ใช้อาจต้อง parse/clean เอง) ข้อดี: ยืดหยุ่นสูง ใส่ data รูปแบบใหม่ลงไปก่อนแล้ว parse ตาม logic ใหม่ทีหลังได้, ไม่มี downtime สำหรับ schema change ข้อด้อย: ความสม่ำเสมอ/คุณภาพข้อมูลไม่ถูกตรวจตั้งแต่แรก, ทำให้การอ่านข้อมูลทุกครั้งต้องทำความสะอาด/แปลงข้อมูลก่อนใช้ อาจเพิ่มความผิดพลาดได้

*Q: คุณรู้จักคำว่า Declarative vs Imperative query หรือไม่? ต่างกันอย่างไร ยกตัวอย่าง A: Declarative query คือการเขียน ระบุแค่ว่า ต้องการข้อมูลลักษณะไหน โดยปล่อยให้ระบบฐานข้อมูลเป็นผู้ตัดสินใจวิธีการดึงข้อมูลเอง (เช่น

SQL) ส่วน Imperative query (หรือ Procedural) คือการเขียน ขั้นตอนการดึงข้อมูลอย่างละเอียด* (เช่นการเขียน loop กรองและรวมข้อมูลในโค้ด) (dev.to)

- ตัวอย่าง: คิวรีด้วย SQL SELECT FROM orders WHERE amount > 100; เป็น Declarative* – ผู้เขียนไม่ต้องระบุว่าจะใช้ index ใดหรือวิธี search แบบไหน แต่ DB Query Optimizer จะวางแผนที่ดีที่สุดเอง ในขณะที่การเขียน loop ใน code เพื่อ scan หา records ที่ต้องการทีละแถวจะถือเป็น imperative
- ข้อดีของ Declarative: ความเรียบง่ายและเพิ่มประสิทธิภาพโดยอัตโนมัติ – โค้ดสั้นและอ่านง่ายกว่า, ฐานข้อมูลสามารถ optimize (เช่น ใช้ Index, ทำ parallelism) ให้อัตโนมัติ โดยนักออกแบบระบบไม่ต้องระบุเองว่า "จะค้นข้อมูลอย่างไร" (dev.to) (dev.to)
- Imperative ยังจำเป็น ในบางกรณีที่เราต้องการความควบคุมแบบละเอียดหรือมี business logic ซับซ้อนที่ภาษา query มาตรฐานอาจรองรับไม่หมด (เช่นการเขียน MapReduce function เอง) แต่ก็ต้องแลกกับโค้ดที่ ซับซ้อนขึ้น และ ดูแลยากกว่า

Q: Omni-Flow แต่ละ Lab เลือกใช้ Data Model และ Query อย่างไรบ้าง? A: ในโปรเจกต์ Omni-Flow มีการใช้หลาย Data Models ตามความเหมาะสม:

- Relational Database: Lab 3 (Debezium CDC) ใช้ PostgreSQL ซึ่งเป็น RDBMS เก็บข้อมูลผู้ใช้/ธุรกรรม ด้วย schema fixed และรองรับ ACID (เหมาะกับ OLTP)
- Columnar Analytical Database: Labs 2, 3 และ 4 ใช้ ClickHouse เป็น Analytical DB (OLAP) ในการ aggregate log/events โดยสร้าง schema ที่ optimize สำหรับ query แบบกลุ่ม/สรุป เช่น Sorting key และ Partitioning ตามช่วงเวลา/ID ใน logs
- Document/Stream Data: ในระบบ event streaming (Kafka) ของทุก Labs ใช้ JSON ข้อความ ใน Kafka ซึ่งมีโครงสร้างยืดหยุ่นสำหรับ events (schema-on-read) แต่เรากำหนดฟิลด์ที่สำคัญและ data type ไว้ล่วงหน้า (ในโค้ด producer/consumer) เพื่อให้ consumer และ sink จัดการง่าย
- หมายเหตุ: ใน Lab 2 (MinIO Lakehouse) มีการแปลง JSON events เป็นไฟล์ Parquet (columnar) เพื่อจัดเก็บบน Object Storage (S3/MinIO) แล้วจึง Query ผ่าน Query engine (เช่น Spark/Trino) ซึ่งเป็นตัวอย่างของ schema-on-read (JSON ดิบ แปลงภายหลัง) ร่วมกับ schema-on-write (Parquet มี embedded schema) ใน architecture เดียวกัน

บทที่ 3: Storage and Retrieval (การจัดเก็บและค้นคืนข้อมูล)

Q: โครงสร้างข้อมูลแบบ B-Tree กับ LSM-Tree ต่างกันอย่างไร? ควรเลือกใช้อย่างไร? A:

- B-Tree Index: เป็น balanced tree แบบปรับแก้ไขบนแผ่นดิสก์โดยตรง (in-place updates) นั่นคือเมื่อแก้ไข record ใดจะเขียนทับตำแหน่งเดิมบนดิสก์เลย ทำให้มี I/O แบบสุ่ม (random I/O) แต่ ช่วยให้อ่าน/ค้นหาได้เร็ว เพราะโครงสร้าง tree ตื้นและจัด key ตามลำดับ สามารถใช้ traversal เล็กน้อยก็เจอข้อมูล (point lookup & range query เร็ว) (medium.com) (medium.com) B-Tree จึงเหมาะกับ workload ที่ อ่านเยอะหรืออ่าน-เขียนสมดุลกัน ที่ต้องการ Latency คงที่ เช่น transactional RDBMS ทั่วไป (MySQL, PostgreSQL เก็บตารางหลักด้วย B-Tree)
- LSM-Tree (Log-Structured Merge Tree): ใช้การเขียนข้อมูลแบบ append-only (เพิ่มต่อท้าย) โดยเขียนลง memtable ก่อนแล้วค่อย flush ลงดิสก์เป็นไฟล์ sorted SSTable (หลายไฟล์) จากนั้น Merge/Compaction รวมไฟล์เล็กเป็นไฟล์ใหญ่ขึ้น เพื่อลบข้อมูลที่ซ้ำ/ล้าสมัยและจัดเรียงข้อมูลใหม่ การเขียนจึงเป็น I/O เชิง sequential ที่เร็วมากและยังบีบอัดข้อมูลได้ดี แต่ การอ่านจะช้ากว่า B-Tree เพราะข้อมูลกระจายหลายไฟล์ ต้องเช็คหลายที่ และอาจต้องรอการ compaction (medium.com) LSM จึงเหมาะกับระบบ เขียนเยอะมาก ๆ และอ่านแบบ scan ครั้งละเยอะ (เช่น log aggregation, time-series db: Cassandra, HBase) ส่วนงานที่ อ่านจุกจิกตลอดเวลาหรือ read-heavy มักใช้ B-Tree ที่ให้ latency การอ่านต่ำกว่าและมี pattern คาดเดาได้มากกว่า (medium.com) (

medium.com)

- การเลือกใช้งาน: ให้ดู profile workload ของระบบ – ถ้า เขียนเยอะ และเขียนทับข้อมูลเดิมไม่บ่อย LSM-tree อาจจะดีกว่าเพราะลดการ random write, แต่ถ้า อ่านบ่อยมาก และต้องการ response ที่เร็วและคงที่ B-tree จะตอบโจทย์กว่า แต่ต้องระวังถ้า workload เขียนเยอะ ๆ B-Tree อาจเกิด write amplification จนประสิทธิภาพตกอย่างมาก (index เยอะ ๆ ทำให้ IOPS ลดลง)

Q: อธิบายความแตกต่างระหว่าง Row-oriented storage กับ Columnar storage และผลต่อ OLTP/OLAP A: ฐานข้อมูล OLTP แบบดั้งเดิม (เช่น RDBMS relational ที่เราใช้ประจำ) ส่วนใหญ่ใช้ row-oriented (เก็บค่าของแต่ละ row ติดกันใน disk page) ทำให้ การอ่าน/เขียนเป็นแถว ทำได้เร็ว (อ่าน 1 แถว โหลดจาก disk เพียง 1 page) เหมาะกับ point lookup/short range เช่น SELECT ตาม PK, แต่เวลาจะอ่านข้อมูลจำนวนมากทั้ง column เพื่อวิเคราะห์ (เช่น SUM, AVG ทั้ง table) จะช้าเพราะ 1 row มีหลาย column ที่เราไม่ต้องการก็ถูกอ่านมาทั้งหมดด้วย ทำให้ I/O สูงมาก (www.systemoverflow.com) (เช่น scan 1 พันล้านแถว x 100 columns = อ่าน 500 GB) ส่วน OLAP database มักใช้ column-oriented (columnar) storage– เก็บข้อมูลแยกตามคอลัมน์ แล้วยังทำ compression ได้ดี (dictionary & run-length encoding) ทำให้การอ่านเฉพาะบางคอลัมน์เร็วขึ้นมาก (เช่น aggregate ยอดขายรายregion ก็อ่านข้อมูลเฉพาะคอลัมน์ Region และ Amount) ลด I/O ลงไปมหาศาล (เช่น 1 พันล้านแถว x 2 columns ที่ต้องใช้ อาจอ่านจริง ๆ แค่ ~10 GB พร้อมบีบอัด) (www.systemoverflow.com) (www.systemoverflow.com) นอกจากนี้ columnar store ยังสนับสนุนการทำ vectorized execution (ประมวลผลค่าเป็นชุดในหนึ่งคำสั่ง) ยิ่งช่วยให้ aggregate query เร็วกว่าการ loop ทีละ row มาก

Q: ทำไมห้ามรัน analytic query หนัก ๆ บน OLTP database? A: เพราะ OLTP DB (row-store) ไม่ถูกออกแบบมาสำหรับการ scan ข้อมูลปริมาณมาก ๆ – หากรัน query สรุปยอดหรือ join หลายตารางขนาดใหญ่ บน production DB จะ กิน I/O และ CPU สูง จนอาจทำให้ query ปกติ (เช่น ค้นข้อมูลทีละ record) ช้าลงมากหรือ time out ไปเลย ซึ่งกระทบผู้ใช้โดยตรง ดังนั้น หลักการสำคัญคือแยก OLAP ออกจาก OLTP ด้วยการทำ ETL/ELT pipeline เพื่อโหลดข้อมูลจาก OLTP (row-store) ไปยังระบบ OLAP (column-store) เป็นรอบ ๆ (batch) เพื่อรัน analytic queries แทน (หรือใช้ DB ที่รองรับ hybrid store ที่มี columnar index สำหรับ analytic, เช่น SQL Server columnstore index) (www.coding-cloud.com) (hld.handbook.academy)

- ตัวอย่าง: Omni-Flow Lab 2 & 3 นำ event logs จาก Postgres (row-based) มาเก็บใน ClickHouse (columnar DB) และลง Parquet บน S3 จากนั้นจึงค่อยใช้ Spark/Trino อ่าน Parquet ใน MinIO เพื่อทำ analytical queries เทียบ performance ของแต่ละแบบ พบว่า analytic query (เช่นการ aggregate ข้อมูลจำนวนมาก) บน ClickHouse/Parquet ทำงานได้เร็วกว่า Postgres อย่างมาก เพราะอาศัยประโยชน์จาก columnar storage และการกระจายงาน

บทที่ 4: Encoding and Evolution (รูปแบบข้อมูลและการเปลี่ยนแปลง Schema)

Q: Forward compatibility และ Backward compatibility คืออะไร? ทำไมจึงสำคัญในระบบที่ส่งผ่านข้อมูลข้าม service? A: ทั้งสองคำนี้เกี่ยวข้องกับ การเปลี่ยนแปลงโครงสร้างข้อมูล (schema evolution)
ในระบบที่มีการสื่อสารระหว่าง service หลายรุ่น:

- Backward compatibility: โค้ด รุ่นใหม่ ต้องอ่าน/ประมวลผลข้อมูลเก่าจากโค้ดรุ่นก่อนหน้าได้ (เช่น consumer รุ่นใหม่ อ่าน message ที่ produce ด้วย schema เก่าได้อย่างถูกต้อง)
- Forward compatibility: โค้ด รุ่นเก่า สามารถอ่านข้อมูลที่สร้างโดยโค้ดรุ่นใหม่ได้ถูกต้อง (เช่น consumer รุ่นเก่าอ่าน message schema ใหม่ได้ แม้มีฟิลด์ใหม่เข้ามา)

การรักษา compatibility พวกนี้สำคัญเพราะในระบบจริงเรา deploy service ทีละส่วน (rolling deploy) ถ้า schema ใหม่ใช้ได้กับรุ่นเก่า จะช่วยให้ zero-downtime (ไม่ต้องหยุดทั้งระบบพร้อมกันเพื่ออัปเกรด) – ในทางปฏิบัติ กฎโดยทั่วไปคือ “การเปลี่ยน schema ต้องไม่กระทบกับการ parse ข้อมูลของ service รุ่นอื่น” เช่น

    - การเพิ่มฟิลด์ใหม่ที่มี default value: มักทำให้ Forward/Backward compatibility ง่าย เพราะระบบเก่าไม่รู้จักฟิลด์นี้ก็จะมองข้ามไป/ใช้ default แทนได้ (ดังตัวอย่าง ถ้าเพิ่มฟิลด์ discount ที่มี default=0.0 ระบบใหม่อ่านไฟล์ Parquet เก่าจะเห็นฟิลด์นี้เป็น 0.0 ได้เลย) (moderndataengineering.dev)
    - การเปลี่ยนหรือลบฟิลด์: มักต้องระวังมาก – เช่น ถ้าลบฟิลด์ออก Backward compatibility จะเสีย เพราะข้อมูลเก่ามีฟิลด์นี้ที่โค้ดใหม่ไม่รู้จัก, หรือถ้าเปลี่ยนชนิดข้อมูล/ความหมายก็อาจทำให้ consume/parse ผิดได้
- แนวทางปฏิบัติ: ใช้เครื่องมือช่วย เช่น Schema Registry สำหรับจัดการ schema เวลาสื่อสารผ่าน Kafka/ฤ gRPC (เช่น Avro/Protobuf) ซึ่งสามารถกำหนด นโยบาย compatibility (backward/forward/full) ได้ล่วงหน้า ช่วยป้องกันผู้พัฒนาส่ง schema ใหม่ที่ไม่เข้ากันเข้าไปในระบบ

Q: JSON กับ Binary format (เช่น Avro/Parquet) มีข้อดี-ข้อเสียอย่างไรในด้านการจัดการ schema? A: JSON อ่านง่าย & ยืดหยุ่นสูง แต่มี ข้อจำกัดด้าน schema เพราะ ไม่มี schema ฝังมากับข้อมูล เลย (schema-on-read เต็มตัว) – ผู้อ่านต้องรู้เองว่าฟิลด์แต่ละอันคืออะไร และ ชนิดข้อมูล ถูก encode เป็น text ทั้งหมด (เช่น "01" อาจจะเป็น string หรือ int ก็ได้) ทำให้ต้อง parse ใหม่ทุกครั้ง และเสี่ยงเจอปัญหา dirty data หรือการตีความผิดๆ ได้ (moderndataengineering.dev) (moderndataengineering.dev) ส่วน Avro/Parquet (Binary + Schema): เก็บ schema ของข้อมูลไว้กับตัวข้อมูลเลย ทำให้เวลาจะอ่าน ระบบรู้ Field และ Type ที่ถูกต้องโดยไม่ต้องเดา ช่วยเรื่อง compatibility เพราะเราสามารถดู version ของ schema ที่ฝังในไฟล์ได้ (ใน Parquet, schema จะอยู่ใน file footer) อย่างไรก็ดี binary format อย่าง Avro/Parquetมี overhead ในการ encode/decode และอ่านยาก (ต้องใช้ tool เฉพาะ) – ในการสัมภาษณ์ถ้าถาม อาจเสริมว่าทีม Data Engineer มักใช้ Avro/Parquet ใน data pipeline เพื่อการันตี schema และประสิทธิภาพการอ่าน แต่ถ้าต้องส่งให้ทีมอื่นใช้หรือ debug ปัญหา ก็มักจะแปลง (export) เป็น JSON/CSV เพื่อความเข้าใจง่าย

Q: Omni-Flow Lab ใดสาธิตเรื่อง schema evolution หรือ encoding? A: Lab 2 (MinIO Lakehouse) ของโปรเจกต์ Omni-Flow แสดงตัวอย่างการใช้ JSON Event ใน Kafka ร่วมกับการเขียนไฟล์ Parquet ลง Data Lake (MinIO) โดยเราจะเห็นว่า ไฟล์ Parquet แต่ละชุดมี schema ของมันเอง – หากเราต้องการ แก้ไข schema เช่น เพิ่ม field ใหม่ใน Event เราต้องวางแผนให้ดี:

- ที่ฝั่ง Kafka (JSON): ถ้าจะเพิ่ม field ใหม่ Producer ควรใส่ค่า default เข้าไป (หรือให้ Consumer ฝั่งเก่า ignore field ที่ไม่รู้จัก)
- ที่ฝั่ง Parquet (Lakehouse): ไฟล์ใหม่ ๆ จะมี field นี้ แต่ไฟล์เก่า ๆ จะไม่มี (ตาม schema เดิม) – Engine ส่วนใหญ่ (Spark, Trino) พออ่านรวมกันจะรวม schema เป็น superset ให้ แต่ การรวม schema หลายรุ่นมี overhead ต้องระวังเรื่องการจัดการ type/compatibility
- Tip: ในระบบจริง ควร เก็บ version ของ schema แต่ละ event/ไฟล์ไว้ (อาจในชื่อไฟล์หรือในคอลัมน์พิเศษ) เผื่อกรณีที่ต้อง debug retroactive ตาม version

บทที่ 5: Replication (การทำสำเนาข้อมูลในหลายโหนด)

Q: Leader-based Replication คืออะไร? และมีแบบ synchronous กับ asynchronous อย่างไร? A: Leader-based replication เป็นรูปแบบที่หนึ่ง Node ใน cluster ถูกเลือกให้เป็น Leader ของข้อมูลชุดหนึ่ง (เช่น topic partition หรือ shard ของ DB) มีหน้าที่ รับการเขียนทั้งหมด แล้ว copy (replicate) ไปยัง Followers (หรือ Secondary replicas) ที่เหลือ

- Synchronous replication: Leader รอ ให้ follower ทุกตัว (หรือบางตัวตามเงื่อนไข) เขียนข้อมูลเรียบร้อยก่อน จึงค่อยตอบ success กลับไป ทำให้ ข้อมูลทนต่อการล่มมากกว่า (เพราะมีหลาย copy แน่ ๆ) แต่ เพิ่ม latency (leader ต้องรอ) และถ้า follower ไม่พร้อมก็อาจ เขียนไม่ได้เลย (availability ลดลง) (www.conduktor.io) (www.conduktor.io)
- Asynchronous replication: Leader ไม่รอ ให้ follower write เสร็จก็ตอบ client เลย – latency น้อยกว่า

และยังเขียนได้แม้ follower บางตัว down (availability สูง) แต่ แลกกับความเสี่ยง data loss ถ้า leader ล่มก่อนที่ follower จะตามทัน (เพราะข้อมูลยังไม่ถูก replicate ไปที่อื่น)

Q: Replication lag คืออะไร? ส่งผลอย่างไรต่อผลลัพธ์การอ่าน? A: Replication lag คือ ความล่าช้าในการตาม ที่เกิดขึ้นเมื่อ follower ยัง sync ข้อมูลไม่ทัน leader – เช่น user เขียนข้อมูล A ไปยัง leader แต่ follower ยังไม่ได้ update A (อยู่ระหว่างตามข้อมูลหรือมีการหน่วง) ดังนั้นถ้ามีการอ่านจาก follower ก่อนตามทัน ก็จะ ไม่เห็นข้อมูล A (stale reads)

- ปัญหา read consistency: ถ้ามีการอ่านจาก follower ที่ lag อยู่ ผู้ใช้อาจ ไม่เห็นข้อมูลล่าสุดหลังการเขียน (ขัดกับหลัก read-after-write consistency) หรืออาจเห็นลำดับข้อมูลที่ไม่ monotonic (เช่น query ครั้งแรกเห็นเวอร์ชันใหม่ แต่ query ครั้งถัดมาไปอ่าน replica ที่ lag กลับเห็นค่าเก่า)
- แนวทางแก้: ถ้าต้องการ read-after-write เสมอ ควร อ่านจาก leader เสมอ สำหรับ key นั้น ๆ หรือใช้ consistency level ที่เข้มขึ้น (เช่นในระบบ NoSQL บางตัว อนุญาตให้กำหนดอ่านแบบ “quorum” ซึ่งรอให้ได้คำตอบตรงกันจากหลาย replica เพื่อกัน read stale) หรือในกรณี Kafka ถ้าต้องการให้ consumer เห็นแต่ message ที่ commit ครบแล้วก็ควรอ่านแบบ read_committed เพื่อข้าม offset ที่ยังไม่ commit

Q: ใน Kafka ค่า acks=all และ min.insync.replicas หมายถึงอะไร? ช่วยป้องกัน data loss ได้จริงไหม? A: คอนเซปต์ acks ของ Kafka คือคำสั่งว่าต้องการให้ Leader รอ การยืนยันการเขียน จาก replicas จำนวนเท่าใดก่อนจะตอบกลับ Producer:

- acks=all หมายถึง Leader จะรอทุก “In-Sync Replica (ISR)” ตอบรับ แล้วจึง reply success กลับ Producer (www.conduktor.io) (โดย ISR = ชุดของ follower ที่ยัง sync ทัน leader ณ ขณะนั้น)
- แต่: acks=all ไม่ได้การันตี replica ทุกตัวใน cluster เขียนข้อมูลลง disk แล้ว เพราะถ้ามี follower ใดหลุดจาก ISR (เช่นช้ากว่ามากหรือ offline ชั่วคราว) Leader จะไม่นับ follower นั้น ทำให้ acks=all อาจเหลือจริง ๆ แค่รอ 1 หรือ 2 ชุด (ขึ้นกับ min.insync.replicas) (www.conduktor.io)
- min.insync.replicas: config ที่กำหนด ขั้นต่ำ ของจำนวน ISR ที่ leader ต้องรอก่อนยอมรับ acks=all – เช่น replication factor=3, ตั้ง min.insync.replicas=2 หมายถึง ต้องมีอย่างน้อย 2 nodes (จาก 3) ที่พร้อมเขียนเสมอ จึงจะเขียนได้; ถ้า online น้อยกว่านั้น (เช่นเหลือ node เดียว) จะเขียนไม่ได้ (Producer จะได้ error NotEnoughReplicasException เลย) (www.conduktor.io)

ด้วยการตั้ง RF=3, acks=all, min.insync.replicas=2 และปิด unclean leader election จะมั่นใจได้ว่า หากมี node ใดล่ม 1 เครื่อง ข้อมูลยังอยู่บน node อื่นอย่างน้อย 1 ชุด (จึงไม่หาย) และระบบยังเลือก leader ใหม่ให้ทำงานต่อได้ทันที แต่ถ้าล่ม เกิน 1 เครื่อง, leader ใหม่จะไม่ถูกเลือก (เพื่อกัน data loss) ทำให้ partition นั้นเขียนต่อไม่ได้จนกว่าจะมี quorum node กลับมา – นี่คือ trade-off ระหว่าง durability กับ availability ใน Kafka ที่เราต้องออกแบบ config ให้เหมาะกับ use case (www.conduktor.io)

Q: เราจะ ป้องกันการอ่านข้อมูลซ้ำซ้อนหรือสูญหาย ในระบบที่มี replication ได้อย่างไร? A:

- ต้องทำให้การเขียนแต่ละครั้ง “commit” บน replica เพียงพอ ก่อนจะอ่าน – เช่น Relational DB ถ้าใช้ single-leader replication แบบ async เราอาจอ่านจาก follower ได้ แต่ไม่ควรใช้กับข้อมูลที่เพิ่งเขียน (เพราะอาจยัง replicate ไม่ทัน) ทางแก้คืออ่านจาก leader จนแน่ใจว่า follower ทันแล้ว หรือยอม trade-off เป็น lagging reads เมื่ออ่านจาก replica
- ใน Kafka เนื่องจาก consumer ส่วนใหญ่ pull data จาก broker ที่เป็น leader ของ partition นั้น ๆ อยู่แล้ว (แปลว่าอ่านของใหม่จาก leader เสมอ) ระบบจึงการันตี monotonic reads ภายใน partition ได้ แต่ถ้า consumer เปิดอ่าน readuncommitted ก็ อาจเห็น message ที่ถูก rollback ได้ (ควรตั้ง isolation.level=readcommitted เพื่อเห็นแต่ที่ commit แล้วเท่านั้น)

Q: ใน Omni-Flow Lab มีการตั้งค่าหรือทดสอบ Replication อย่างไร? A: ใน omni-command-center (lab หลัก)

เราใช้ Kafka cluster 3 nodes และตั้ง replication factor = 3 ทุก topic, พร้อม config min.insync.replicas=2 และ acks=all ใน Producer (ตามหลัก “หากต้องการ durability สูง ควรให้ RF=3 และ min.insync=2 (www.conduktor.io)) – จากนั้นเราทดสอบ scenario ที่ leader ล่ม โดย kill Kafka broker ของ leader ของ partition ดูว่าระบบจะเลือก leader ใหม่จาก ISR ที่เหลือ โดยไม่สูญเสีย message หรือไม่ (ตาม config unclean leader election ที่กำหนด) ซึ่งผลการทดสอบพบว่า ระบบหยุดรับการเขียนชั่วคราว ระหว่างเลือก leader ใหม่ และ consumer จะรออ่านจนกว่าจะมี leader ใหม่ แต่ ไม่มี event ใดสูญหาย (High-watermark ของ Kafka รับประกันว่าข้อมูลที่ commit แล้วจะไม่หาย) อีกการทดลองคือปรับลด min.insync.replicas ให้เหลือ 1 (เทียบเท่า acks=1) แล้ว kill leader – พบว่า message ชุดล่าสุดก่อน leader ล่มสูญหาย เพราะเขียนไม่ทัน replicate ไป node อื่น (นี่คือเหตุผลว่าควรใช้ min.insync=2 ในการ deploy production และหลีกเลี่ยง unclean leader election)

บทที่ 6: Partitioning (การแบ่งพาร์ติชันข้อมูล)

Q: ในระบบอย่าง Kafka ทำไมการรับประกันลำดับ (ordering) ถึงจำกัดเฉพาะภายใน partition เดียว? A: Kafka (เช่นเดียวกับระบบกระจายอื่น ๆ) เลือกการแบ่งพาร์ติชันเพื่อขยายระบบ (scale out) ซึ่งต้องแลกมากับการสละลำดับรวมของทุก event – สถาปัตยกรรม Kafka กำหนดว่า Producer เลือก partition key เพื่อกำหนดว่า event นั้นจะถูกส่งไปเก็บที่ partition ใด และ Kafka รับประกัน ลำดับของ message ภายในแต่ละ partition เท่านั้น (message ใน partition เดียวเรียงตาม offset) แต่ระหว่าง ข้ามคนละ partition ลำดับจะไม่ถูกการันตี เพราะ event อาจมาถึงแต่ละ partition ไม่พร้อมกัน (เช่น event A/B คนละ key เขียนลงคนละ partition ก็อาจสลับเวลากันได้) ในการออกแบบ ถ้าเราต้องการลำดับที่เคร่งครัด (strict ordering) ของ event ทั้งหมด (global order) เราจำเป็นต้องส่ง event ทั้งหมดไปที่ partition เดียว ซึ่ง จะกลายเป็นคอขวด ของระบบ (throughput จำกัดเพียงเท่าประสิทธิภาพเครื่องเดียว) – ดังนั้น แนวทางที่นิยม คือ ยอมแตก partition เพื่อให้ scale ได้ แล้วคุมการสั่งงาน/ลำดับในขอบเขตเล็ก แทน เช่น คำสั่งโอนเงินในบัญชีเดียวกัน อาจกำหนด key = หมายเลขบัญชี เพื่อให้ทุกรายการที่เกี่ยวกับบัญชีนี้เข้าพาร์ติชันเดียว (จึงคงลำดับ relative กัน) ส่วนระหว่างต่างบัญชีไม่จำเป็นต้องมี global order เดียวกัน ซึ่งช่วยให้เพิ่มจำนวน partition (เพิ่ม throughput) ได้นั่นเอง

Q: Hot spot / Skew คืออะไร? และเราจะแก้ปัญหา hot partition อย่างไร? A: Hot spot หรือ hot partition หมายถึงสถานการณ์ที่ข้อมูล/ทราฟฟิกกระจุกตัวอยู่ที่ พาร์ติชันเดียวหรือไม่กี่พาร์ติชัน เท่านั้น ทำให้โหนดที่ถือพาร์ติชันนั้น มีโหลดสูงกว่าตัวอื่นอย่างมาก เกิดคอขวดจนแย่ง resource ของ cluster (www.vetoralabs.com) (www.vetoralabs.com) – สาเหตุอาจมาจาก key ที่ “ดัง” มาก (celebrity key) เช่น รหัสลูกค้ารายหนึ่งถูก request มากผิดปกติ, หรือ การ partition ผิดหลักการ เช่น เลือก key ที่ ความหลากหลายต่ำ (low cardinality) อย่าง state/country code ทำให้ทุก record ไปกองอยู่ไม่กี่พาร์ติชัน เทคนิคแก้ Hot key/hot partition:

- Key salting: เติม suffix บางอย่างเข้าไปใน key (เช่น user123#0, user123#1, ..., user123#N) เพื่อ กระจาย traffic ของ key เดียวออกเป็นหลายพาร์ติชัน (คล้ายการ shard ภายใน key) (www.vetoralabs.com) วิธีนี้ตรงไปตรงมาที่สุดและ เพิ่มความสามารถในการเขียนแบบเส้นตรงกับจำนวน salt factor N (N=10 อาจรองรับ load ได้ ~10 เท่า) แต่ต้อง แลกกับความยุ่งยากในการอ่าน – ระบบที่ต้องรวมข้อมูลตาม key เดิมต้องอ่านจาก ทุก salt partition แล้วรวมผลลัพธ์ (ทำให้ Query ยากขึ้นและ latency เพิ่มขึ้นตามจำนวน salt factor) (www.vetoralabs.com)
- Dynamic Partitioning/Splitting: ในระบบฐานข้อมูลบางประเภท (โดยเฉพาะ NoSQL หรือ distributed NewSQL) เราสามารถ ตรวจจับพาร์ติชันที่ร้อนจัด แล้ว แบ่งพาร์ติชันนั้นออกเป็นสอง (หรือย้าย key บางส่วนไปอีกพาร์ติชัน) เพื่อกระจายโหลดออก – วิธีนี้ซับซ้อนกว่า key salting แต่ ไม่กระทบลำดับการอ่านภายในแต่ละพาร์ติชัน (เพราะยังเป็น key เดิมในพาร์ติชันใหม่) เพียงแต่ ต้องมีเมตาดาต้าช่วยบอกที่อยู่ partition ของ key แต่ละช่วง (complex routing)
- Caching / Read Replicas: ในกรณี Hot reads (key ใดมีคนอ่านซ้ำ ๆ จำนวนมาก) อาจใช้วิธี Cache ผ่อนโหลด

(เช่น Redis cache หน้า DB) หรือทำ read replicas ให้หลายโหนดช่วยกันตอบการอ่าน key นั้น – วิธีนี้ ไม่ช่วยถ้า load เป็นการเขียน และต้องยอมให้ข้อมูลที่อ่านออกมาอาจ stale บ้าง (dirty read) แต่ช่วยให้ระบบโดยรวมไม่ล่ม

Q: Omni-Flow Labs มีการออกแบบ partition key อย่างไร และได้ทดสอบ hot spot ไหม? A: ใช้ตัวอย่างจาก omni-command-center (lab หลักอีกครั้ง):

- เริ่มแรกเรา partition data ด้วย device_id (อุปกรณ์) เพื่อให้ทุกข้อความจากอุปกรณ์เดียวกันไปที่ partition เดียวกัน (รักษาลำดับภายใน device) – แต่เราจำลอง Hot Spot โดยตั้งค่าให้ 90% ของ event ทั้งหมดมาจาก device ตัวเดียวกัน (เช่น device “P0”) ผลคือ partition เดียวของ P0 มี message มากกว่าพาร์ติชันอื่นจน consumer lag สูงขึ้น และ latency สูง (เพราะ consumer ที่กิน partition นั้นช้ากว่าเพื่อน)
- จากนั้นเราลองใช้ Key Salting กับ device P0 (เช่น สร้าง 3 คีย์: P0#0, P0#1, P0#2 แทนที่จะใช้ P0 เดียว) – ผลคือ event ของ P0 ถูกกระจายไปหลาย partition ทำให้ โหลดสมดุลขึ้น, latency โดยรวมดีขึ้น แต่ก็ต้องเขียนโค้ด consumer ให้รวมข้อมูล P0 จากหลายสาย (เช่น aggregate ข้าม 3 partition เพื่อหาผลรวมจริงของ P0) เพิ่มความซับซ้อนขึ้นมา ซึ่งก็เป็นตัวอย่าง trade-off real-world ของการแก้ hot key

บทที่ 7: Transactions (ธุรกรรมของข้อมูล)

Q: คุณสมบัติ ACID transactions มีอะไรบ้าง? แต่ละข้อหมายถึงอะไร? A: ACID เป็น acronym ที่ใช้อธิบายคุณสมบัติของ Transaction ในระบบฐานข้อมูลแบบ relational (หรือระบบอื่น ๆ ที่รองรับ atomic transaction):

- Atomicity (ความเป็นหน่วยเดียว): ธุรกรรม (transaction) จะต้องเกิดขึ้นทั้งหมด หรือไม่เกิดเลย – ถ้ามีการทำงานใดใน transaction ล้มเหลว ระบบต้อง rollback กลับสู่สถานะก่อนเริ่ม transaction ทั้งหมด
- Consistency (ความสอดคล้องของข้อมูล): หลัง transaction สำเร็จ ข้อมูลในระบบต้องอยู่ในสถานะที่ถูกต้องตามกฎ business และ constraints ทั้งหมด (เช่น ไม่ละเมิด unique constraint หรือ sum balance ไม่ติดลบ ฯลฯ) – database engine มักช่วย enforce ตรงนี้ เช่น ตรวจ constraints ตอน commit transaction
- Isolation (การแยกการทำงานพร้อมกัน): เมื่อหลาย transaction ทำงานพร้อมกัน ต้องไม่รบกวนกัน – ผลลัพธ์สุดท้ายจะต้อง เหมือนกับ แต่ละ transaction ทำงานทีละอันเรียงกัน แม้ว่าจริง ๆ จะรันพร้อมกัน (concept: serializability) ซึ่ง DBMS บางชนิดอาจไม่ได้ให้ isolation สมบูรณ์ (เช่น default Isolation = Read Committed อาจเกิด phenomena อย่าง phantom reads) แต่ยิ่ง isolation สูงจะยิ่งลด concurrency และ performance
- Durability (ความทนทานของข้อมูล): เมื่อ transaction บันทึกสำเร็จ (committed) แล้ว ข้อมูลที่เขียนจะไม่สูญหายแม้ระบบล่ม – DB จะ flush log/พยุง transaction log ไว้ใน disk ที่ทนทาน (เช่นใช้ Write-Ahead Log - WAL ก่อนยืนยัน commit)

Q: Isolation Level ของ Transaction มีกี่แบบ? ต่างกันอย่างไร? A: ANSI SQL กำหนด 4 ระดับ isolation (จากต่ำไปสูง):

1. Read Uncommitted: สามารถอ่าน uncommitted data ของ transaction อื่นได้ (เกิด dirty reads, non-repeatable reads, phantom reads* ได้หมด) – มักไม่ค่อยใช้จริงเพราะความสอดคล้องต่ำมาก
2. Read Committed: (ค่า default ของหลาย DB) – ไม่อ่าน data ที่ยังไม่ commit ของ transaction อื่น (ไม่มี dirty reads) แต่ยังเกิด non-repeatable reads (อ่านซ้ำสองครั้งได้ค่าต่าง) และ phantom (จำนวน row ใน result set เปลี่ยนระหว่างสองการอ่านในเดียวกัน) ได้
3. Repeatable Read: (default ของ MySQL InnoDB) – ล็อก (หรือ MVCC pin) ทุก row ที่อ่านไว้จนจบ transaction ดังนั้นอ่านซ้ำ ๆ จะได้ค่าเท่าเดิม (ไม่มี non-repeatable) แต่ ยังเกิด phantom reads* ได้ (เพราะอาจมีการแทรก row ใหม่ที่ตรง filter ระหว่าง transaction)
4. Serializable: Isolation สูงสุด – ทำให้ ธุรกรรมทั้งหมดมีลักษณะเหมือนทำงานต่อ ๆ กันทีละตัว (serial

schedule) โดยป้องกัน phantom reads ด้วยการใช้ predicate locks หรือ validation ตอน commit – ให้ correctness สูงสุดแต่ แลกกับ throughput ที่ต่ำลง อย่างมากถ้า workload มีการเขียน/อ่านซ้อนกันเยอะ ๆ

Q: อธิบาย Two-Phase Commit (2PC) แบบเข้าใจง่าย และข้อจำกัดของมัน A: 2PC เป็นโปรโตคอลมาตรฐานที่ใช้ทำ Distributed Transaction (ข้ามหลาย database/service) ให้เป็น atomic:

- Phase 1 (Prepare): Coordinator (ตัวกลาง) จะส่งคำสั่ง Prepare ไปให้ทุกบริการ/DB ที่เกี่ยวข้อง แต่ละที่ทำการจอง resource หรือเขียน log ไว้ก่อน แล้วตอบกลับว่า “Prepared” (หรือ “Abort” ถ้าเตรียมไม่สำเร็จ)
- Phase 2 (Commit): ถ้า ทุก node ตอบพร้อม – Coordinator จะส่ง Commit ไปทุก node ให้ทำการ commit; ถ้ามี node ใดตอบ Abort หรือไม่ตอบ (timeout) – Coordinator จะส่ง Abort (rollback)

ข้อดี: ทำให้แน่ใจว่าทุกระบบ commit หรือยกเลิกพร้อมกันทุกที่ (Atomicity) ข้อเสีย: 2PC *มีปัญหา Blocking** – หาก Coordinator ล่ม หลังจากสั่ง Prepare แล้ว แต่ยังไม่ทันสั่ง Commit/Abort, ทุก node จะค้าง (block)* รอคำสั่งต่อไปโดยยกเลิกไม่ได้ (Lock resource ค้าง) (adevguide.com) เพราะแต่ละ node ไม่รู้สถานะสุดท้ายว่าควร commit หรือ abort (เนื่องจากการตัดสินใจ commit/abort อยู่ที่ Coordinator ที่หายไป) – ต้องรอจนกว่า Coordinator เดิมฟื้นหรือมี mechanism พิเศษมาแก้ (เช่น timeout & manual intervention หรือ Three-Phase Commit (3PC) ที่พยายามลดปัญหานี้ แต่ก็เพิ่ม overhead มาก) ในระบบไมโครเซอร์วิสปัจจุบัน นิยมเลี่ยง 2PC เพราะซับซ้อนและ scale ลำบาก – หันไปใช้ eventual consistency และ pattern อย่าง Saga (ลำดับของ local transactions + compensating actions กรณีล้มเหลวแต่ละขั้น), หรือ Transactional Outbox (อธิบายด้านล่าง) เพื่อหลีกเลี่ยงการต้อง lock distributed resources นาน ๆ

Q: ถ้าต้องเขียนข้อมูลทั้งลงฐานข้อมูล และส่ง event เข้า Kafka พร้อม ๆ กัน (dual write) – คุณจะออกแบบให้ operation นี้เป็น atomic ได้อย่างไร? A: ปัญหา dual-write ที่ต้องอัปเดต 2 ระบบพร้อมกัน (เช่น DB + Kafka) โดยปกติจะ ไม่มี transaction ข้ามสองระบบ (unless ใช้ 2PC/3PC ซึ่งอย่างที่กล่าวไป มีความเสี่ยงสูง) – หนึ่งในแนวทางมาตรฐาน สำหรับ microservices คือ Transactional Outbox pattern (developer.confluent.io) (developer.confluent.io):

- ขั้นตอน: แทนที่จะเขียน Kafka โดยตรงจาก Business Service เราให้บริการนั้น เขียน event ลง “Outbox Table” ใน DB เดียวกับข้อมูล business พร้อมกับการ commit ธุรกรรมหลัก (เป็นการทำ local transaction เพียงครั้งเดียว) (developer.confluent.io)
- จากนั้นใช้อีก process หนึ่ง (เช่น job เล็ก ๆ หรือใช้ Debezium ซึ่งเป็น Change Data Capture tool) คอยอ่าน Outbox table และ publish event ที่ค้างใน Outbox ไป Kafka – เมื่อส่งสำเร็จแล้วก็อัปเดต flag หรือ ลบ record ใน Outbox
- วิธีนี้ทำให้มั่นใจได้ว่า state ใน DB และ event ใน Kafka สอดคล้องกันเสมอ (ถ้า business data commit ไม่ผ่าน event ก็ไม่ถูกเขียนเข้า Outbox) และเราสามารถ ใส่ Retry logic ในส่วนส่ง Outbox Kafka เพื่อการันตี at-least-once (ส่งซ้ำได้) จนกว่า event จะถึง Kafka แน่นอน (developer.confluent.io) (developer.confluent.io) (รับได้ถ้ามี event ซ้ำและ handle ที่ปลายทางด้วยการ deduplicate อีกชั้น)
- ตัวอย่าง: Omni-Flow Lab 3 (Debezium CDC) ใช้แนวคิด outbox นี้ – แอปพลิเคชันเขียนธุรกรรมลง Postgres และ Debezium tail ที่ WAL log ของ Postgres เพื่อดึง ทุกการเปลี่ยนแปลง มาส่งเข้า Kafka (ในฐานะ event) ต่อไป ClickHouse – วิธีนี้ทำให้ ตารางใน Postgres กับ Kafka topic มีข้อมูลตรงกันรายการต่อรายการ (ถ้าเขียนลง Postgres ได้ Event ก็ถูกส่งออก Kafka แน่นอน) โดยไม่ต้องไปแตะ 2 ระบบในโค้ดแอปโดยตรง

บทที่ 8: The Trouble with Distributed Systems (ปัญหาในระบบกระจายศูนย์)

Q: ทำไมการออกแบบ Distributed System จึงยากกว่าระบบเดี่ยว? Partial Failure คืออะไร? A: ในระบบแบบกระจาย (distributed) เรามักเจอ partial failure คือ การที่ส่วนหนึ่งของระบบล่มหรือขัดข้อง ขณะที่ส่วนอื่นยังทำงานอยู่ (

university.kurz.fyi) – เช่น เซิร์ฟเวอร์หนึ่งใน cluster ล่ม, network ระหว่าง 2 service ขาด, หรือ process บางตัวหยุด (เช่น GC pause) ในขณะที่ตัวอื่นยังวิ่งปกติ ผลคือ ระบบโดยรวมอาจยังดูทำงานอยู่ แต่บางฟังก์ชันเพี้ยนหรือช้าลง ในขณะที่ single-server system ถ้าพังคือพังทั้งระบบ ชัดเจนกว่า

- ปัญหาเพิ่มเติม: ในระบบกระจาย เครือข่าย (Network) เชื่อถือไม่ได้ 100% – การไม่ตอบสนองของ service หนึ่ง เราไม่อาจรู้แน่ชัดว่า เกิดจาก (ก) ปลายทางล่มจริง หรือ (ข) แค่ตอบช้า/เครือข่ายติดขัดชั่วคราว (university.kurz.fyi) – เราต้องใช้ Timeout ในการเดาเอาเองว่าควรถือว่าล่มเมื่อใด แต่ตั้งเวลาสั้นไปก็ไป kill service ที่แค่ช้า, ตั้งยาวไปก็รอนานเกินเมื่อ service ล่มจริง
- Clock & Time: แต่ละเครื่องมี นาฬิกา (clock) ของตัวเองที่ไม่ sync เป๊ะกัน – เวลาของ event ที่ stamp มากับข้อมูลอาจไม่สอดคล้องกับเวลาจริงที่ process ประมวลผล (ดูคำถาม event time ด้านล่าง)
- Process Pauses: OS threads/process อาจหยุดชั่วคราว (เช่น GC Stop-the-world 100 ms หรือ Container ถูก preempt) โดยที่ system อื่นไม่รู้ แต่ทำให้ระบบเสียความต่อเนื่อง – เช่น Leader ที่มี lease 30s หากหยุด process 20s อาจทำให้ หมดอายุความเป็น leader โดยไม่รู้ตัว แต่ฟื้นมาทำงานต่อ ก่อข้อมูลขัดแย้งกับ leader ใหม่ (เรียกว่า split-brain) (university.kurz.fyi) (university.kurz.fyi)

Q: At most once, At least once, Exactly-once delivery semantics แต่ละแบบหมายถึงอะไร? A: ทั้ง 3 คำนี้เป็นการรับประกันการส่งข้อมูลในระบบที่มี asynchronous messaging:

- At most once: ข้อความอาจสูญหายได้ แต่จะไม่เกิดการซ้ำ – ระบบแบบนี้ ไม่การันตี ว่าทุก message จะถึงปลายทาง แต่ถ้าถึงจะมาถึงแค่ครั้งเดียว (ตัวอย่าง: UDP, หรือ Kafka ที่ producer ใช้ acks=0 จะได้ latency ต่ำมากแต่ message อาจหายได้) (insightful-data-lab.com)
- At least once: ข้อความไม่มีวันสูญหาย แต่มีโอกาสถูกส่งซ้ำ – ระบบอาจส่งข้อความเดิมมากกว่าหนึ่งครั้ง แต่อย่างน้อยต้องส่งถึงหนึ่งครั้งแน่นอน (เช่น JMS, Kafka (default)) – การส่งซ้ำอาจเกิดจากการ retry เมื่อ network ล่มแล้วไม่แน่ใจว่าข้อความก่อนหน้าส่งสำเร็จไหม เป็นต้น (insightful-data-lab.com) (insightful-data-lab.com)
- Exactly once: ข้อความแต่ละอันถูกประมวลผล “ครั้งเดียวเท่านั้น” – ฟังดูดีที่สุด แต่ในทางปฏิบัติต้องระบุ ขอบเขต ให้ชัด: เช่น Kafka เคลมว่ามี exactly-once ได้เมื่อ ภายใน Kafka ecosystem (เช่น Kafka Streams + Connect ที่ commit offset และผลลัพธ์ใน transaction เดียวกัน ทำให้ consumer ที่อ่านแบบ read_committed จะไม่เจอข้อความซ้ำ) (insightful-data-lab.com) แต่ ถ้านอกระบบ Kafka (เช่นเขียนข้อมูลไปฐานอื่นนอก Kafka) เรายังต้องจัดการ idempotency/deduplication เอง อยู่ดี – หลัก ๆ แล้ว exactly-once มักหมายถึง “effectively exactly once” ที่ผลลัพธ์สุดท้ายถูกต้องเหมือนประมวลแต่ครั้งเดียว (แม้เบื้องหลังอาจ retry หลายครั้ง) (developer.confluent.io)
- สรุป: ระบบ distributed ส่วนใหญ่เลือก guarantee ระหว่าง at-most, at-least เพื่อ balance ระหว่าง performance vs correctness – ถ้าต้องการ exactly-once แบบ end-to-end ต้องออกแบบโครงสร้างเพิ่ม (เช่น distributed transaction หรือ idempotent consumer) ซึ่งเพิ่มความซับซ้อนสูง

Q: ระบบคุณจัดการกับ การส่งซ้ำและการส่งไม่ครบ (duplicate & lost messages) อย่างไร? A: (คำถามนี้ผู้สัมภาษณ์ต้องการดูว่าเรารู้จักเทคนิคแก้ปัญหาในระบบกระจายหรือไม่) – หลัก ๆ มี 2 กรณีคือ กัน message ซ้ำ กับ กัน message หาย:

- ป้องกัน/ตรวจจับ Duplicate:
  - *ใช้ IDempotency: กำหนด Unique ID ให้แต่ละ event แล้วให้ฝั่ง consumer เก็บ seen ID เพื่อตรวจจับและข้าม (discard) event ที่รับเข้ามาซ้ำ (เช่น database insert ใช้ primary key เดิมซ้ำจะ conflict เอง หรือถ้าโปรแกรมวางแผนไว้ อาจมี idempotency key* เปรียบเทียบ)
  - การ merge โดยใช้ key: เช่นใน Omni-Flow Lab 3 ตอนเขียนข้อมูล CDC ลง ClickHouse เราใช้ Engine ReplacingMergeTree โดยกำหนด primary key และใส่ version timestamp ทำให้ถ้า Debezium ส่ง event

update ของ row เดิมมาซ้ำ (snapshot + realtime events) ClickHouse จะคงเหลือ record สุดท้ายไว้ชิ้นเดียว (update แทนที่ record เดิม) – อย่างไรก็ตาม การใช้ engine แบบนี้ ต้องระวัง: ถ้าจะอ่านข้อมูลที่ merge แล้วแบบ ฉบับรวมผลลัพธ์สุดท้าย ใน ClickHouse อาจต้องใช้คำสั่ง FINAL ที่มี overhead เข้ามาช่วย (ไม่อย่างนั้นจะเห็น record ซ้ำระหว่างยัง merge ไม่เสร็จ)

- ป้องกัน Data Loss:
  - Persist data ใน durable store*: pipeline ที่สำคัญต้อง ensure ว่า event ถูกเขียนลง disk ที่ทนทาน (เช่น Kafka commit log, DB WAL, cloud storage) ก่อนคอนเฟิร์ม – ใน lab4 เราออกแบบ Stream Ingester ให้เขียน event ทุกตัวลง Bronze storage (S3/MinIO)* ทันทีที่ได้รับ เพื่อเก็บ raw data ที่ replay ได้เสมอ
  - Monitoring & Alerting: ต้องมีระบบ Data Monitoring ตรวจจับความผิดปกติ เช่น consumer lag สูง (ส่อให้เห็นว่ามี event ค้างในคิว), error rate, หรือ สถิติการนับ record ระหว่างระบบที่ไม่ตรงกัน (อธิบายเพิ่มในหัวข้อ Data Reconciliation ด้านล่าง)

Q: ใน Omni-Flow Lab 4 (Orchestration & Audit) มีการจัดการคุณภาพข้อมูลอย่างไร? A: Lab 4 เป็นตัวอย่างของ Data Pipeline แบบมี Data Quality & Reconciliation – เราแบ่งข้อมูลออกเป็น 3 ชั้น (Bronze/Silver/Gold) โดยมีการทำ Data Quality Gate ระหว่างชั้น:

- Bronze (Raw landing): รับ event ดิบจาก Kafka เขียนลง storage (Parquet บน MinIO) ทุก event (full log)
- Silver (Base table): เราใส่ ขั้นคัดกรองคุณภาพข้อมูล – ใช้สคริปต์ stream_ingester.py อ่าน Bronze batch ละช่วงเวลา แล้วทำ validations (เช่น schema validation, completeness check ฯลฯ) จากนั้น แยกข้อมูลเป็น 2 ทาง: event ที่ผ่านเกณฑ์จะไปยัง Silver (table หลัก), ส่วนที่ ไม่ผ่าน (invalid) จะถูกใส่ Quarantine (เช่น table แยกอีกตารางหนึ่ง) พร้อมเหตุผลที่ไม่ผ่าน
- Gold (Data Mart): ใช้ orchestration (Airflow/DAG ใน orchestrator_app.py) ดึงข้อมูลจาก Silver มาประมวลผลรวม (เช่น summary, ML features) ได้เป็น Data Mart พร้อมใช้เชิงธุรกิจ – ที่สำคัญคือมีขั้น Reconciliation ตรวจสอบว่า ข้อมูลที่มาถึง Gold ครบถ้วนไม่สูญหาย โดยเทียบ สัดส่วน/ยอดรวม กับ Silver/Bronze ตาม expected logic (เช่นผลรวมธุรกรรมใน Gold ควรตรงกับผลรวมธุรกรรมใน Silver ในช่วงเวลาเดียวกัน) – ถ้าไม่ตรงก็ให้ trigger แจ้งทีมงานทันที

บทที่ 9: Consistency and Consensus (ความสอดคล้องของข้อมูลและการทำ Consensus)

Q: Eventual Consistency คืออะไร? A: Eventual consistency คือ model การรับประกันแบบอ่อน (weak guarantee) ในระบบ replicate/กระจายข้อมูล ที่ระบุว่า ถ้าไม่มีการเขียนใหม่เข้ามา และระบบทำงานไปสักระยะ t (ไม่กำหนดตายตัว) สุดท้ายแล้วทุก replicas จะมีข้อมูลตรงกัน (blog.codefarm.me) – พูดอีกอย่างคือ ระบบจะ “converge” สู่สถานะที่สอดคล้องกันในท้ายที่สุด แต่ *ไม่มีการการันตีว่าเมื่อใด (ระหว่างนี้ client อาจอ่านค่าเก่าบ้างใหม่บ้างก็ได้) ซึ่ง trade-off ของ eventual consistency คือ ยอมความไม่สอดคล้องชั่วคราว เพื่อแลกกับ ความเร็วและความทนทานระบบ* (เช่นเขียนแล้วตอบกลับเร็ว, รองรับ Partition Tolerance ได้ดีกว่า strong consistency)

Q: อธิบายคำว่า Linearizability (Strong Consistency) คืออะไร? A: Linearizability คือ model การสอดคล้อง (consistency model) ที่เข้มงวดมาก ที่สุดรูปแบบหนึ่ง โดยมองว่าข้อมูล ราวกับมีสำเนาเดียวในระบบ (one-copy semantics) – เมื่อ client ใดเขียนค่าหนึ่งสำเร็จ client ทุกตัวที่อ่านข้อมูลเดียวกันหลังจากนั้นต้องเห็นค่านั้นทันที (blog.codefarm.me) กล่าวคือระบบ ต้องตอบค่าที่ใหม่ล่าสุดเสมอ (recency guarantee) ไม่มีโอกาสเห็น ค่า stale แม้มี replication ก็ตาม

- ในทางปฏิบัติ การจะทำให้ linearizable ได้ มักหมายถึงการที่ระบบต้องมี ศูนย์กลางจัดลำดับเหตุการณ์ (เช่น single leader และ อ่านจาก leader เสมอ) หรือใช้ Distributed Consensus (เช่น Raft/Paxos) เพื่อให้ ทุก node

เห็นคำสั่ง (log) ชุดเดียวกันตามลำดับเดียวกัน – สิ่งที่ต้องแลกมาคือ latency ที่สูงขึ้นและ readiness ของระบบที่อาจลดลง (เช่นถ้ารอฟัง มติเอกฉันท์จากทุก node ก็ต้องแลกกับการหยุดชะงักเมื่อบาง node ไม่พร้อม) (blog.codefarm.me) (blog.codefarm.me)

Q: Distributed Consensus คืออะไร? มีความสำคัญอย่างไร? A: Consensus คือ การทำให้หลายโหนดในระบบกระจาย “เห็นพ้อง” กับค่าบางอย่าง ปกติประยุกต์ใช้ในการเลือกผู้นำ (leader election) หรือการบันทึก state กลาง (Atomic broadcast of log) – ปัญหา consensus นั้นแก้ยากเพราะเงื่อนไขของระบบกระจาย (network ไม่เสถียร, โหนดล่ม, partial failures) แต่มีการคิดค้นอัลกอริทึมที่มี พิสูจน์ทางทฤษฎี เช่น Paxos และ Raft ที่สามารถทำให้กลุ่มโหนดตกลงเลือกค่าหนึ่งร่วมกันแม้บางส่วนล่มไป ได้ (เช่น โหวตเลือก leader หรือ commit transaction) – การมี consensus เป็นพื้นฐานให้เราสร้าง property “เหมือนระบบมี 1 copy” บน cluster หลาย node ได้ (เป็นพื้นฐานของ strong consistency หลายแบบ)

- ตัวอย่าง: Kafka KRaft mode (v.2.8+) – Kafka เลิกใช้ ZooKeeper และหันมาใช้กลุ่ม Controller (metadata) quorum 3+ nodes ทำ Raft consensus เพื่อเก็บ metadata ของ cluster (เช่น รายชื่อ topic, broker, partition leaders, ISR ต่าง ๆ) อย่าง ทนทานและสอดคล้องกัน – Controller ที่ชนะโหวตจะเป็น active controller คอยจัดการ metadata changes, ส่ง event การเปลี่ยน leader/ISR ให้ทุก broker อัปเดตตาม (metadata ถูกเก็บใน Kafka _clustermetadatabus เหมือน log) (kafka.apache.org)
- อย่างไรก็ตาม *อย่าสับสนระหว่าง metadata consensus (KRaft) กับ การ replicate ข้อมูลในแต่ละ topic ของ Kafka – แม้ KRaft จะใช้ Raft-protocol ในการจัดการ metadata แต่ การ replicate event ของแต่ละ partition ยังทำผ่านกลไก Leader-Follower ไม่ใช่ consensus เต็มรูปแบบ (Leader ตัดสินใจลำดับ ส่ง replicas, ข้อมูล commit เมื่อเข้า ISR quorum แล้ว) – ผลคือ Kafka ยังคงได้ throughput สูง กว่าระบบที่ใช้ consensus กับทุก event แต่ก็ อาจเกิดกรณีข้อมูลหายได้เล็กน้อย* ถ้าหากไม่ตั้งค่า ACK/quorum ให้เหมาะสม (ดูบทที่ 5 เรื่อง acks)

Q: CAP Theorem ที่ว่า “Consistency, Availability, Partition Tolerance เลือกได้สองในสาม” คุณเข้าใจว่าอย่างไร? A: CAP Theorem เป็นหลักการเชิงทฤษฎี (จาก Eric Brewer) ที่อธิบาย trade-off ในระบบกระจาย: เมื่อเกิด Network Partition (P) – ระบบ ไม่สามารถมีได้ทั้ง Consistency (C) และ Availability (A) พร้อมกัน ต้องเลือกอย่างใดอย่างหนึ่ง

- เลือก Consistency (เสีย Availability): เช่น Database แบบ distributed ที่ใช้ consensus (Raft) – พอ Partition เกิดขึ้น Node ส่วนน้อยจะหยุดให้บริการ (เพื่อกันข้อมูลไม่สอดคล้อง) เหลือแต่ฝั่งข้างมาก (quorum) ที่ยังตอบสนอง – ผู้ใช้ฝั่งที่ minority partition จะใช้งานไม่ได้จนกว่า network จะกลับมา
- เลือก Availability (เสีย Consistency): เช่น ระบบอย่าง DNS, Internet routing หรือบาง NoSQL (Cassandra ใน eventual mode) – เมื่อ Partition เกิดขึ้น ทุก Partition ยังตอบได้ปกติ (availability สูง) แต่ข้อมูลอาจ inconsistent ระหว่าง Partition – ต้องค่อย sync หรือ reconcile กันภายหลัง (eventual consistency)

บทที่ 10: Batch Processing (การประมวลผลแบบกลุ่ม)

Q: ทำไม Batch processing ยังสำคัญในยุค Big Data? มีข้อดีอย่างไรเมื่อเทียบกับ real-time? A: Batch processing เน้นการประมวลผลข้อมูล ขนาดใหญ่ ที่มี ขอบเขตแน่นอน (bounded data) – เช่น ประมวลผลข้อมูลรายวัน/รายชั่วโมง หลังจากที่รวบรวมข้อมูลครบแล้ว ทำให้มี ข้อดี:

- ผลลัพธ์ทวนซ้ำได้ (reproducible): เราสามารถ เก็บ input เดิม แล้ว run batch ซ้ำเมื่อไหร่ก็ได้ และเพราะข้อมูล immutable ไม่เปลี่ยนแปลงระหว่างรัน batch (สถานะชัด–Input เท่าเดิมทุกครั้ง) ทำให้ผลลัพธ์ deterministic และ debug ได้ง่ายกว่า streaming ที่ input เกิดขึ้นเรื่อย ๆ
- รองรับขั้นตอนซับซ้อนและหนัก: งาน batch มักเป็น pipeline หลายขั้นที่ใช้เวลานาน (เช่น ETL data ทำ Machine Learning training) – Batch frameworks (Hadoop, Spark) ถูกสร้างมาให้ ทนทานต่อการล้มเหลวระหว่างทาง (มี checkpoint/retry) และ optimize การอ่าน/เขียน large dataset แบบ sequential ได้ดี

- ปรัชญาการออกแบบ: Process ข้อมูลจำนวนมหาศาลที่ immutability เพื่อให้ผลลัพธ์ reproducible ได้เสมอ โดยอาศัยการแบ่งงานเป็นขั้นตอนที่ deterministic, ขอบเขตข้อมูลที่แน่นอน และการจัดการ fault-tolerance (เช่น Spark lineage) ซึ่งช่วยให้ batch pipeline มีความน่าเชื่อถือ ควบคุมง่าย (ankit-rathi.github.io)

Q: Lambda architecture คืออะไร และยังจำเป็นไหมในปัจจุบัน? A: Lambda architecture เป็นแนวคิดการออกแบบ Data System โดยแยกการประมวลผลข้อมูลเป็น 2 ทางพร้อมกัน:

- Batch layer: เก็บ Immutable master dataset ทั้งหมด (append-only, ไม่แก้ไขอดีต) แล้วประมวลผลสร้าง materialized views (เช่น batch table ที่ aggregated แล้ว) อย่าง ถูกต้องสมบูรณ์ (เพราะใช้ข้อมูลครบทุกชิ้น ไม่พลาดแม้ event มาช้า)
- Speed/Streaming layer: ประมวลผล เฉพาะข้อมูลใหม่แบบ real-time เพื่อให้ได้ผลลัพธ์ทันทีที่ข้อมูลมา (แต่ข้อมูลอาจยังไม่สมบูรณ์ เช่น event มาช้าอาจถูกมองข้ามไป) จากนั้นเมื่อ batch ทำเสร็จ (เช่น ทุกเที่ยงคืน) ก็รวมผล กับ speed layer เพื่อให้ได้ข้อมูลสุดท้ายที่ครบถ้วน

แนวคิดนี้ช่วยแก้ปัญหาข้อมูลล่าช้า (late data) ใน streaming โดยการให้ batch มาตามแก้ทีหลัง แต่ ข้อเสีย คือระบบซับซ้อนมาก (ทำ 2 ส่วนซ้ำซ้อนกัน) – ปัจจุบันบางระบบจึงไปทาง Kappa architecture (ทำ streaming อย่างเดียว ปล่อยให้ event ใดมาช้าก็ compute เมื่อมา แล้ว streaming engine ปรับผลย้อนหลัง) แต่ Kappa ก็มีข้อจำกัดต้อง design ให้ระบบรองรับ out-of-order events ดี ๆ (ดู event time/watermark)

Q: Omni-Flow Lab 4 (Orchestration & Audit) ใช้หลักการ batch อย่างไร? A: Lab 4 ใช้แนวคิด batch pipeline ชัดเจนดังนี้:

- มี ตาราง Bronze/Silver/Gold ใน Delta Lakehouse ซึ่ง Bronze (raw) และ Silver (cleaned) เป็น immutable append-only (เก็บตามวัน/ชั่วโมง) – ทำให้ทุกครั้งที่ต้องแก้ logic ในการประมวลผล เราสามารถ reprocess ย้อนหลัง โดยการอ่านข้อมูล Bronze/Silver ที่เก็บไว้ทั้งหมดมา run batch job ใหม่ ให้ได้ผลลัพธ์ Gold ที่ถูกต้อง (เช่นเมื่อเปลี่ยนวิธีนับ metric, เรารัน pipeline ใหม่จากข้อมูลเริ่มต้นเพื่อสร้างผลลัพธ์ใหม่ แล้วค่อยเขียนทับ Gold table เดิม) – technique นี้ใช้จัดการทั้ง backfill ข้อมูลเก่า และ Data lineage เพื่อ audit ว่าผลลัพธ์ (Gold) มาจาก input อะไร
- มีการใช้ Airflow (orchestrator) ในการจัดการ job dependency (เช่น Gold รันได้ต้องรอ Silver เสร็จก่อน) และจัดการ scheduling/retry (เช่น ถ้า batch ล่มให้รันใหม่)
- Pipeline ใน Lab 4 ยังมี Data Quality checks ก่อนจะ propagate data ไปสู่ขั้นต่อไป – ถ้าเจอ batch ไหนที่ตัวเลขผิดปกติ (เช่น KPI ผิด threshold) จะ หยุด pipeline ไว้ก่อนที่จะโหลดเข้า data warehouse (fail fast with alert) เพื่อป้องกันไม่ให้ข้อมูลผิดไหลไปสู่ downstream reports

บทที่ 11: Stream Processing (การประมวลผลข้อมูลแบบสตรีม)

Q: Event time ต่างจาก Processing time อย่างไร? ทำไมการแยกสองอย่างนี้ถึงสำคัญในระบบสตรีม? A:

- Event time: เวลาจริงที่เหตุการณ์เกิดขึ้นตามข้อมูลที่แนบมากับ event (เช่น timestamp ที่ device gen ข้อมูลหรือเวลาบันทึกลง DB)
- Processing time: เวลาที่ระบบประมวลผล event นั้น ๆ (อิงตามนาฬิกาของเครื่องที่รันโปรแกรม) (developer.confluent.io)

ความต่างนี้สำคัญเพราะในระบบสตรีม มักเจอปัญหา event มาถึงไม่เรียงลำดับกันตามเวลาจริง (out-of-order) – ถ้าเราใช้ processing time อย่างเดียว เช่น window 1 นาที นับจำนวน event โดยอิงนาฬิกาเครื่องประมวลผล เหตุการณ์ที่มาช้า (มาทีหลังแต่เวลาจริงเก่า) จะ ไม่ถูกนับใน window ที่ควรอยู่ (สูญหาย) หรือถ้านับไปก็จะ นับผิดช่วงเวลาได้ – Flink และระบบสตรีมสมัยใหม่จึงรองรับ Event time processing โดยให้เรากำหนด field เวลาใน record และใช้ Watermark เพื่อบอกระบบว่าควรรอนานแค่ไหนสำหรับ event ที่อาจมาช้า ก่อนปิด window หรือเลื่อน state

ไปขั้นถัดไป (developer.confluent.io) (developer.confluent.io)

- ตัวอย่าง: ถ้ามีนโยบาย Watermark = “สมมุติว่า event อาจมาช้าสุดไม่เกิน 5 นาที” – Flink จะคอยใส่ watermark ลง stream เป็นระยะ เมื่อเวลาประมวลผลเกิน 5 นาทีจาก event ล่าสุดที่เห็น ก็จะ ถือว่า window ของช่วงเวลาก่อนหน้าครบถ้วนแล้ว (no more late events) แล้วปล่อยผลลัพธ์ออกมาได้, แต่ถ้าหลังจากนั้นยังมี event เก่ากว่า 5 นาที pas เข้ามา ระบบจะ mark ว่า late event และเราต้องกำหนดว่าจะจัดการยังไงกับ late event (เช่น ทิ้งไปหรืออัปเดตผลลัพธ์ย้อนหลัง) (developer.confluent.io) (developer.confluent.io)

Q: ระบบสตรีมให้ Exactly-once processing ได้จริงหรือ? ระบบอย่าง Flink/Kafka ทำงานอย่างไร? A: Apache Flink รองรับ stateful streaming โดยใช้กลไก Checkpoint/Snapshot – โดยหลัก Flink จะคอย snapshot state เป็นช่วง ๆ ลง durable storage (เช่น S3) แล้วใช้ two-phase commit protocol คู่กับ Kafka เพื่อกัน event หายหรือซ้ำ: เมื่อ checkpoint สำเร็จ Flink จะ commit transaction ของ Kafka sink (ที่ opened ไว้ตั้งแต่ก่อน snapshot) ทำให้ทุก operator ก่อนหน้าจนถึง sink commit งานที่ snapshot นั้นพร้อมกัน – หาก node ไหนล่ม Flink จะหยุดทั้ง job แล้ว rollback state ของ operator ทุกตัวกลับไปยัง snapshot ล่าสุด จากนั้น reset offset ของ Kafka กลับไปจุด checkpoint แล้วค่อย run ต่อ ทำให้ event ที่ประมวลผลไปแล้วก่อน snapshot นั้นจะถูกประมวลผลซ้ำ แต่เพราะ Kafka ใช้ระบบ transactional producer + read_committed มันจะ ไม่เกิดผลลัพธ์ซ้ำซ้อน (ข้อความเดิมถูก rollback ไปก่อนแล้ว) (developer.confluent.io) (developer.confluent.io) – ด้วยวิธีนี้ Flink + Kafka จึงเคลมว่าให้ ”Exactly-once (effectively)” จาก ต้นทาง Kafka ถึงผลลัพธ์ใน Kafka sink ได้จริง (หรือพูดอีกแบบคือ อย่างมากก็ส่งผลหนึ่งครั้ง)

- ข้อควรระวัง: ขอบเขต exactly-once ของ Flink+Kafka ข้างต้น จำกัดอยู่ภายใน pipeline นั้น – ถ้า output ของ Flink เขียนไปที่ฐานข้อมูลหรือระบบอื่นนอก Kafka ยังต้องการกลไกตรวจสอบ idempotency/transaction ที่ปลายทาง (เช่น upsert โดยมี primary key, หรือใช้ outbox ของ DB) มาช่วยเสริมความเป็น end-to-end exactly once อยู่อีกชั้นหนึ่ง (โดยทั่วไป ไม่มีระบบกระจายไหน ให้การันตี exactly-once ข้าม ทุกองค์ประกอบ ได้สมบูรณ์หากไม่มีการออกแบบเพิ่มเติม)

Q: Omni-Flow Lab 1 (Flink Stream) และ Lab 3 (Debezium CDC) สาธิตการประมวลผลสตรีมอย่างไร? A:

- Lab 1 (Flink Streaming): ใช้ Flink SQL (ผ่าน Flink Table API) สร้าง streaming job ที่อ่านธุรกรรมการชำระเงินจาก Kafka (จำลองเหตุการณ์เข้ามาเรื่อย ๆ) แล้วทำ filter (กรองธุรกรรมที่ผ่านเกณฑ์) จากนั้นทำ aggregation แบบ tumbling window 5 นาที เพื่อคำนวณสรุปยอด – ตัวอย่างนี้ชี้ให้เห็นการใช้ event time (timestamp ในข้อความ) เป็นตัวจัด window และ Flink ใช้ watermark 5s กำหนดให้รอ event ที่มาช้าไม่เกิน 5 วินาที (ให้ real-time เพียงพอ) หากเกินนั้นถือว่า late (ระบบยังเก็บลง Kafka ได้แต่มี flag “late” มา)
- Lab 3 (Debezium CDC): ใช้ Debezium (Change Data Capture) อ่าน WAL log ของ Postgres (OLTP) แบบเรียลไทม์ เปลี่ยนแต่ละ database commit เป็น Kafka event (โดย Debezium จะแปลง insert/update/delete ให้เป็น event พร้อม metadata เช่น op-code, transaction id, LSN offset) – จากนั้น cdcsinkworker.py จะคอยอ่าน event พวกนี้จาก Kafka แล้วเขียนลง ClickHouse (OLAP) – วิธีนี้ช่วยสร้าง stream pipeline ที่ replicate ข้อมูลจาก OLTP ไป OLAP แทบเรียลไทม์ (หลักวินาที) แทนการใช้ batch ETL – ความท้าทาย: การจัดการ ลำดับ และ การส่งซ้ำ – Debezium ส่ง event แบบ at-least-once (เช่นบางครั้ง snapshot + realtime update ซ้ำรายการเดียวกัน) จึงต้องเลือก engine & key ที่เหมาะสมใน OLAP (ใน lab เราใช้ ReplacingMergeTree และ primary key ที่เหมาะสม เพื่อคงเหลือข้อมูลสุดท้ายชิ้นเดียว) แต่ทั้งนี้ ClickHouse ก็ไม่การันตี exactly-once ครบทุกกรณี (เช่น ถ้า Crash ระหว่าง merge อาจมี duplicated fragment ได้ ต้องใช้ query FINAL เข้ามาช่วยลบ)

บทที่ 12: The Future of Data Systems (อนาคตของระบบจัดการข้อมูล)

Q: เมื่อข้อมูลเดียวกันถูกจัดเก็บหรือใช้งานในหลายระบบ (เช่น database, data warehouse, index ฯลฯ) จะเกิดปัญหาอะไร? เราจะทำให้ข้อมูลสุดท้าย “ถูกต้องและสอดคล้องกัน” ได้อย่างไร? A: ปัญหาที่เจอบ่อยคือ

ข้อมูลไม่สอดคล้องกันข้ามระบบ – เช่น ระบบ A มี record มากกว่า/น้อยกว่าระบบ B (ทำให้รายงานคนละที่แสดงไม่ตรงกัน), หรือข้อมูลในระบบหนึ่งอัปเดตไปแล้วแต่อีกระบบยังเก่า (timing ต่างกัน), และการแก้ไขข้อมูลในหลายที่ทำให้เกิด “ชุดข้อมูลจริง” หลายชุด (เช่น one source of truth หายไป มีหลาย source แทน) แนวทางแก้ไขหลัก ๆ:

- ออกแบบ Data Pipeline และ Integration: ให้มี จุดกำเนิดข้อมูลเดียวที่ชัดเจน (single source of truth) – เช่น ใช้วิธี Change Data Capture (CDC) เพื่อกระจายการเปลี่ยนแปลงจาก OLTP ไปยัง data warehouse แทนการเขียนแยกสองที่, หรือใช้ event bus รวมศูนย์ (เช่น Kafka) ที่ทุกระบบรับ event จากแหล่งเดียวกัน
- *ทำ Data Reconciliation & Audit**: หลังวางระบบแล้ว ต้องมีการตรวจสอบความสอดคล้องของข้อมูลระหว่างระบบอย่างสม่ำเสมอ เช่น รายวันหรือสร้าง automated job* – วิธีทั่วไปคือ เปรียบเทียบจำนวน records/ยอดรวม ของแต่ละชุดว่าตรงกันหรือไม่ ถ้าไม่ตรงควรสืบสวนทันที (insightful-data-lab.com) (github.com) นอกจากนี้ การทำ key-level reconciliation (join ระหว่างชุดข้อมูลเพื่อตรวจ keys ที่ขาด/เกิน) จะช่วยระบุรายการที่สูญหายหรือซ้ำซ้อนอย่างแม่นยำ เช่นรายงาน Missing records (มีในต้นทางแต่ไม่มีในปลายทาง), Extra records (เกินมาในปลายทาง), หรือ Mismatch (มีครบแต่ค่าต่างกันเกิน threshold) (github.com)
- ปรับปรุง Data Quality ที่ต้นทาง: นอกจากตรวจสอบ หลังจากส่งข้อมูลแล้ว ควรมีการสร้าง Data Quality rules สำหรับ validate ข้อมูลก่อนเข้า pipeline ด้วย เช่น ตรวจ schema, ความครบ (NOT NULL), ความสัมพันธ์ (FK constraints), หรือ business rule (เช่น enddate ต้องไม่ก่อน startdate) – ข้อมูลที่ไม่ผ่านเกณฑ์คุณภาพควรถูกจัดเก็บแยก (quarantine) และแจ้งเตือนไปยังผู้เกี่ยวข้อง เพื่อแก้ไขที่ต้นทางหรือพิจารณาเอาเข้าระบบในขั้นตอนพิเศษต่อไป

Q: คุณมีประสบการณ์การทำ Data Audit/Reconciliation ตรวจสอบข้อมูลในระบบจริงอย่างไรบ้าง? A: ใน Omni-Flow Lab 4 (Orchestration & Audit) เราแสดงตัวอย่างการทำ Data Reconciliation ไว้อย่างชัดเจน – โดยหลังจาก pipeline รันจบในแต่ละรอบ Orchestrator จะทำการ คำนวณตัวตรวจสอบ (verification) เพื่อเทียบข้อมูล: เช่น Table Gold (ผลลัพธ์สรุป) ที่สร้างจาก Silver ควรจะมี จำนวนแถวหรือผลรวมยอด ที่สอดคล้องกับ Silver หรือ Bronze (raw data) ในช่วงเวลาเดียวกัน (เช่น Bronze = Silver + Quarantine) ถ้าพบว่า จำนวน row ไม่เท่ากัน หรือยอดรวมผิดไปเกิน threshold ที่ยอมรับ จะถือว่า pipeline ผิดปกติและต้อง alert เพื่อให้เราสืบสวนต่อ

- เทคนิคที่ใช้: เราจะรัน SQL reconciliation queries โดยใช้ LEFT JOIN ระหว่าง tables เพื่อหาว่า keys ไหนบ้างที่หายไป (missing) หรือ มาเกิน (extra) หรือ จำนวนแตกต่าง (เช่น order ยอดเงินไม่ตรงกันเกิน ±X ตาม tolerance ที่ตั้ง) แล้วสรุปออกมาเป็น รายงานข้อแตกต่าง (discrepancies report) (github.com) – จากการทดสอบพบว่าการจำแนกประเภทความต่างชัด ๆ (missing vs extra vs mismatched) มีประโยชน์มาก เพราะสาเหตุและแนวทางแก้ต่างกัน (เช่น missing มักแปลว่าพลาดส่ง event, extra หมายถึงส่งซ้ำ หรือ mismatched หมายถึงคำนวณผิด) (github.com) ทำให้ทีมรู้ว่าจะตามแก้ที่ตรงไหน
- การออกแบบ pipeline ให้ตรวจสอบได้: นอกจากนี้ Lab 4 ยังเก็บ log การรันแต่ละรอบลง Audit table และ มี record ID กลาง ให้แกะรอย event ตั้งแต่ Bronze Silver Gold(data lineage) – เวลาเจอ discrepancy เราจึงสามารถไล่ย้อนดู event ต้นทางได้ง่าย (ตรวจสอบว่า event ตกหล่นในขั้นไหน) – สิ่งเหล่านี้คือหลักการ Observability ของ data pipeline ที่สำคัญในงาน Data Engineer

---

ประเด็นเชิงลึกสำหรับตอบคำถามติดตาม (Deep-Dive Explanations & Defenses)

นอกจากคำตอบพื้นฐานข้างต้น ผู้สัมภาษณ์ระดับ Senior มักคาดหวังให้เราขยายความในแง่ trade-offs และความเสี่ยง/ข้อจำกัด ของเทคนิคแต่ละอย่างได้อย่างชัดเจน – ด้านล่างนี้เป็นประเด็นเชิงลึกที่ผู้สมัครควรเข้าใจเพื่อใช้

โต้ตอบ หรือขยายความเมื่อต้องตอบคำถามระดับสูง

- แยกแยะคุณสมบัติของระบบอย่างชัดเจน: Reliability (ความถูกต้องแม่นยำของการทำงาน), Availability (ความพร้อมให้บริการ), Scalability (การรองรับการขยาย), Consistency (ความสอดคล้องกันของข้อมูล), Durability (ความคงทนของข้อมูล) ฯลฯ – แต่ละคำมีนิยามต่างกัน อย่าใช้ปะปนกัน เช่น “ระบบเราทนทาน 100%” ควรแจกแจงว่าทนทาน (durable) ระดับไหน, availability เท่าไร, consistency ระดับใด (strong/eventual) เป็นต้น
- *ระบุขอบเขตของ “Exactly-Once”**: หากอ้างว่า “ระบบส่งข้อมูลแบบไม่ซ้ำ 100%” ผู้สัมภาษณ์อาจถามจี้ – ต้องอธิบายว่า exactly-once นี้ แค่ไหน เช่น ภายในระบบ Kafka เอง (ใช้ Kafka transactions + read_committed) หรือ ครอบคลุมทั้งระบบถึงปลายทางจริง ๆ* (ซึ่งมักต้องออกแบบเพิ่มเอง) (insightful-data-lab.com) อย่าเคลมเกินจริงโดยไม่มีที่มารองรับ
- *ออกแบบ Partition & Key อย่างระวัง: Partition ช่วย scale ระบบแต่ทำให้ การันตีลำดับได้เฉพาะใน scope เล็กลง – ถ้า จำเป็นต้องคงลำดับของ entity (เช่นการทำงานของอุปกรณ์ IoT) ควรใช้ key ที่เจาะจง entity นั้น แต่ถ้า key นั้นทำให้เกิด hotspot* มากเกิน (เช่น load 80-90% ไปกองที่พาร์ติชันเดียว) ต้องมีวิธี mitigate เช่น key salting (www.vetoralabs.com), partitioning แบบอื่น หรือการ caching ไม่งั้นจะได้ scalability แต่ระบบโดยรวมจะช้าลงจากคอขวด
- *แยกเรื่อง Metadata consensus (KRaft) vs Data replication: ใน Kafka, การใช้ Raft (KRaft) นั้น เพื่อให้ metadata (ข้อมูลควบคุม cluster)* ถูกจัดการอย่างสอดคล้อง แต่ การ replicate event ในแต่ละ partition ยังคงใช้ leader-based (ไม่ใช่ consensus) – ตรงนี้ผู้สัมภาษณ์อาจคาดหวังให้เราอธิบายความแตกต่าง (Raft ใช้เลือก controller + จัดการ metadata, ส่วน actual data ได้ consistency/availability จาก acks & ISR) (kafka.apache.org)
- พิสูจน์การทดสอบด้วยข้อมูลที่เป็นรูปธรรม: หากจะพูดถึงตัวเลข performance (เช่น “ระบบรองรับ 5,000 events/s โดย p99 latency < 1s”) ควรเสริม วิธีที่เราได้ตัวเลขนี้มา (เช่น จากการ load test ใน lab ไหน, config อย่างไร, วัดบน environment แบบไหน) ไม่ควรตอบเป็นตัวเลขลอย ๆ
- ยอมรับข้อจำกัดและสิ่งที่ไม่ได้ทำ: สุดท้าย การตอบแบบ Senior คือการ รู้ว่าระบบเราไม่เพอร์เฟ็กต์ – ต้องกล้าอธิบาย ข้อจำกัด ของ design เราเองและ สิ่งที่ยังไม่ได้ implement พร้อม แนวทาง mitigate เช่น “Kafka acks=all + min.insync=2 ทำให้ถ้า broker ล่ม 1 ตัวข้อมูลไม่หาย แต่ถ้าล่ม 2 ตัวพร้อมกัน ระบบเราจะเขียนไม่ได้; เราแก้ด้วยการ alert ทีมทันทีและมี HA policy ให้ spin-up broker ใหม่เร็วที่สุด” – การแสดงความเข้าใจข้อจำกัดเหล่านี้และมีแผนรองรับจะช่วยให้คำตอบเราน่าเชื่อถือและดูเป็น Architecture decision ที่รอบคอบ

หมายเหตุท้าย: คู่มือนี้จัดทำเพื่อสรุปและเชื่อมโยงความรู้จากทฤษฎีสู่การปฏิบัติ
ผู้สัมภาษณ์อาจต่อยอดถามรายละเอียดเพิ่มเติมได้ ดังนั้นการเข้าใจภาพรวมทั้งหมดและฝึกตอบโดยใช้ภาษาที่กระชับ ชัดเจน และตรงประเด็น (พร้อมยกตัวอย่างจากประสบการณ์จริง) จะทำให้คุณโดดเด่นในการสัมภาษณ์ตำแหน่ง Senior Data Architect

แหล่งข้อมูล

[1] Designing Data-Intensive Applications Discussion Questions
Reading: bookey.app
แหล่งที่มา: bookey.app

[2] Reliability and Fault Tolerance: Faults vs. Failures — System Design
Reading: university.kurz.fyi
แหล่งที่มา: university.kurz.fyi

[3] Chapter 2: Designing Data-Intensive Applications - Data models - Part 2 ...
Reading: dev.to
แหล่งที่มา: dev.to

[4] B-tree vs. LSM-tree: Understanding Their Differences - Medium
Reading: medium.com
แหล่งที่มา: medium.com

[5] Storage and Access Optimization: Row vs Column, Indexes, and ...
Reading: systemoverflow.com
แหล่งที่มา: systemoverflow.com

[6] Database Architecture: OLTP vs OLAP Separation
แหล่งที่มา: coding-cloud.com

[7] OLTP vs OLAP: Row Stores, Column Stores, and Matching Shape to Workload
แหล่งที่มา: hld.handbook.academy

[8] Choosing a format: Parquet vs Avro vs ORC (and schema evolution) | Modern Data Engineering Guide
Reading: moderndataengineering.dev
แหล่งที่มา: moderndataengineering.dev

[9] Kafka min.insync.replicas Explained | Conduktor
Reading: conduktor.io
แหล่งที่มา: conduktor.io

[10] Hotspot Mitigation -- Partitioning | System Design Concepts — Vetora
Reading: vetoralabs.com
แหล่งที่มา: vetoralabs.com

[11] Two-Phase Commit: Protocol, Blocking and Failure Modes
Reading: adevguide.com
แหล่งที่มา: adevguide.com

[12] The Transactional Outbox Pattern - Confluent
Reading: developer.confluent.io
แหล่งที่มา: developer.confluent.io

[13] The Trouble with Distributed Systems: Partial Failures, Clocks, and Pauses — System Design
Reading: university.kurz.fyi
แหล่งที่มา: university.kurz.fyi

[14] Pipeline Correctness: Duplicates and Loss | Insightful Data Lab
Reading: insightful-data-lab.com
แหล่งที่มา: insightful-data-lab.com

[15] Exactly-Once Processing in Apache Flink
Reading: developer.confluent.io
แหล่งที่มา: developer.confluent.io

[16] Designing Data-Intensive Applications: Consistency and Consensus
Reading: blog.codefarm.me
แหล่งที่มา: blog.codefarm.me

[17] KRaft vs ZooKeeper | Apache Kafka
Reading: kafka.apache.org
แหล่งที่มา: kafka.apache.org

[18] Batch Processing
Reading: ankit-rathi.github.io

แหล่งที่มา: ankit-rathi.github.io

[19] Event Time and Watermarks - Confluent

Reading: developer.confluent.io

แหล่งที่มา: developer.confluent.io

[20] Data Quality & Reconciliation Framework - GitHub

Reading: github.com

แหล่งที่มา: github.com

[21] upgrad.com

Reading: upgrad.com

แหล่งที่มา: upgrad.com

[22] github.com

Reading: github.com

แหล่งที่มา: github.com

[23] redpanda.com

Reading: redpanda.com

แหล่งที่มา: redpanda.com

[24] notes.stonecharioteer.com

Reading: notes.stonecharioteer.com

แหล่งที่มา: notes.stonecharioteer.com

[25] confluent.io

Reading: confluent.io

แหล่งที่มา: confluent.io